

University Enrollment System

The Brief: what it is, how it is built, and where it is weak

Explainer document for Salman Adnan

The project in one paragraph

A desktop course-registration application for a university student: log in with an email and password, confirm a six-digit code sent by email, then browse a catalogue of course sections, put some in a cart, enroll, drop, swap between sections, view the resulting weekly timetable, export it to a spreadsheet, mark attendance, and read a log of everything you did. Built with PyQt6 for the windows and SQLAlchemy over a single SQLite file for the data. Roughly 2,570 lines of Python across 13 modules, 11 screen layouts, and one pre-seeded database. It was the semester project for the Databases course at Habib University, built by a two-person team.

This document covers the whole project at altitude. It assumes no prior knowledge, but it puts the definitions in a glossary at the end rather than interrupting the argument. Where it says something is verified, that means it was checked by running the code or by opening the shipped database, not by trusting the README.

Two things carry the project and get proper treatment here: the **data model** (Section 2) and the **enrollment rules** (Section 3).

1 The architecture in one picture

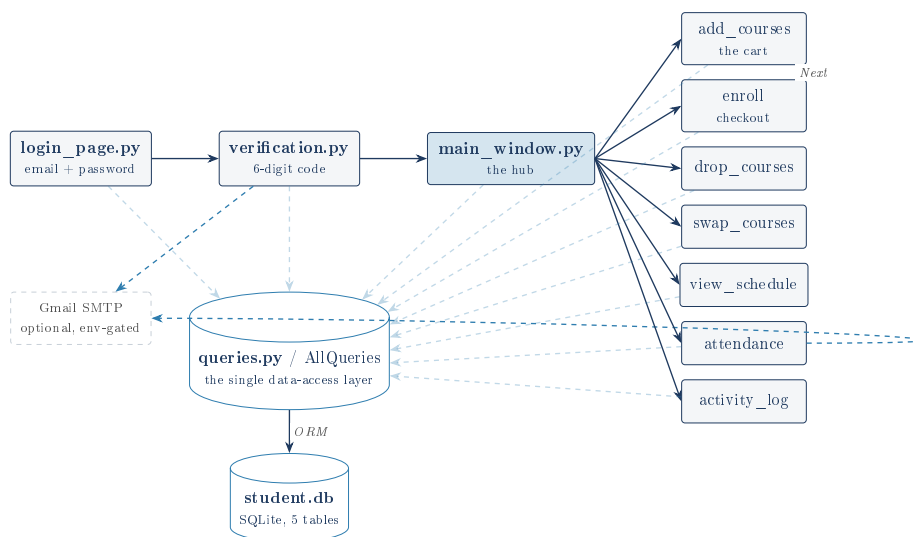


Figure 1: Every screen is a `QMainWindow` subclass paired with a Qt Designer `.ui` layout loaded at runtime. Solid arrows are window transitions; faint dashed arrows are data-access calls.

The one architectural decision that matters

No window ever writes a database query. Every screen, without exception, goes through `AllQueries` in `queries.py`. The README says this was deliberate: with two people each owning different screens, one data-access layer was the contract between them, so the UI on either side only had to agree on method names. The code bears that out, and it is the

project's best structural idea.

The cost is that `AllQueries` is a 410-line class of about 25 loosely related methods with no internal structure, and its correctness depends on an instance attribute (`glob_id`) that every screen has to remember to set.

2 The data model

This is a Databases course project, so the schema is where the marks were. It is five tables in one SQLite file. Everything below was verified against the shipped `student.db`, not read off the README.

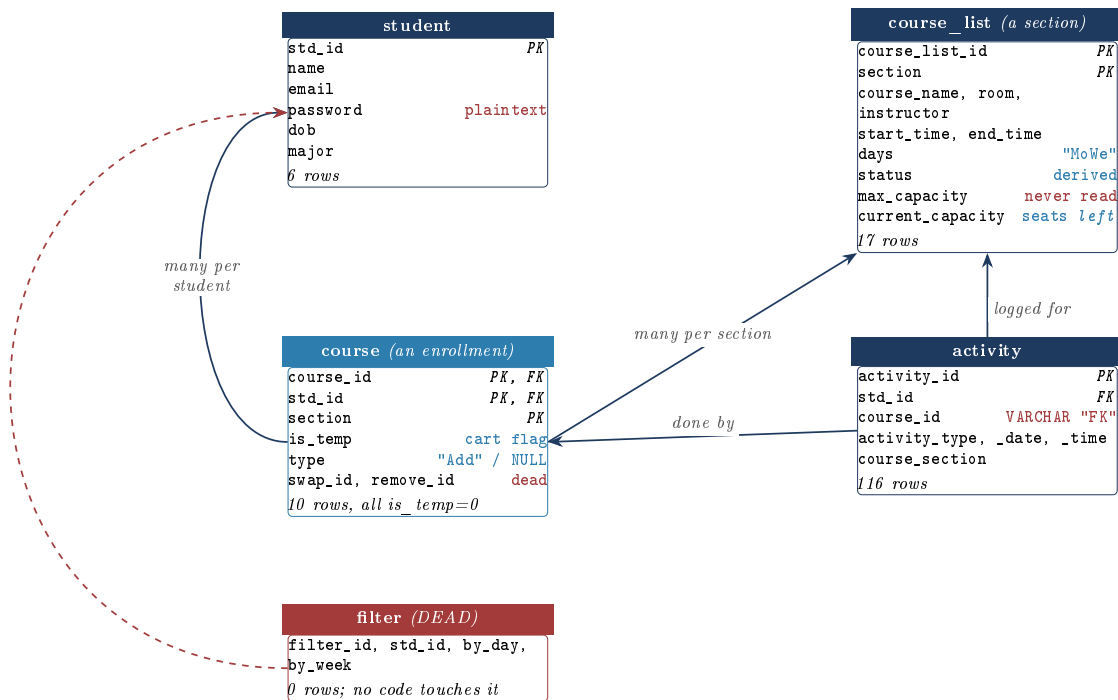


Figure 2: The schema, with row counts read out of the shipped `student.db`. Blue is the junction table that expresses the whole enrollment idea. Red marks what is declared but dead.

2.1 The one idea that unlocks the schema

A row in `course_list` is a *section*, not a course

`course_list` has a composite primary key, (`course_list_id`, `section`). So “Data Structure II” is three rows, not one: 201/L1 on MoWe at 11:30, 201/L2 on WeFr at 14:30, 201/L3 on MoWe at 16:00. The course name and every timing detail is repeated on each.

Everything else follows. A student enrolls in a section. A clash is between sections. A swap, as Section 3 shows, is defined as moving between two sections of the same course. And the `course` table is a **junction table**: its key (`course_id`, `std_id`, `section`) is exactly the sentence “this student holds this section of this course, at most once”.

The junction table does double duty, and this is the second idea worth carrying:

is_temp	type	meaning
True	"Add"	sitting in the cart, no seat taken
False	NULL	really enrolled, a seat is taken

2.2 What is wrong with the schema

The foreign keys are structurally invalid, and are never enforced anyway

Two independent facts compose into the most serious schema finding, and both were verified by running SQLite against a copy of the shipped file.

First, SQLite does not check foreign keys unless asked. Unlike most databases, its `PRAGMA foreign_keys` defaults to off, and nothing in this project turns it on. Verified: inserting a `course` row for a student who does not exist, referencing a course that does not exist, is accepted silently. Every `ForeignKey` in the model is runtime documentation.

Second, you cannot simply turn it on. `course.course_id` and `activity.course_id` both reference `course_list.course_list_id`, which is only *half* of that table's composite primary key and is therefore not unique (id 201 appears three times, once per section). A foreign key must target a unique column set. Enable the pragma and SQLite refuses the schema outright:

```
1 PRAGMA foreign_keys = ON;
2 PRAGMA foreign_key_check;
3 -- OperationalError: foreign key mismatch - "activity" referencing "course_list"
```

The same mistake appears twice more, on the dead `swap_id` and `remove_id` columns. The root cause is one thing: composite keys were used correctly for the primary keys and then only half of them was carried into the foreign keys. The fix is a table-level `ForeignKeyConstraint` over (`course_id`, `section`) instead of a column-level `ForeignKey`. This is the finding to be ready for, because it is the exact subject of the course this was submitted to.

The catalogue is not normalised

Because a row is a section, “Data Structure II” is stored three times. Rename the course and three rows must change together or the catalogue contradicts itself. The textbook split is two tables, a `course` (id, name) and a `section` referencing it with the timing, room, instructor, and capacity. An examiner will find this in ten seconds. The honest answer is that the denormalisation buys nothing here except simpler queries, and the split is what you would do now.

The activity log is prose in a column that claims to be a foreign key

`activity.course_id` is declared a `String` with a `ForeignKey` to an integer primary key, and it holds, verified in the shipped data: `'201'`, `'201, 202'`, `'201, 202, 205, 310, 330'`, `'201 -> 201'`, the literal four-character string `'None'`, and real NULLs. It is a human-readable audit trail rendered straight into a table widget, not a queryable relation. You cannot ask “how many times was 201 dropped” without parsing strings.

The `'None'` strings are a nice detail: current code writes a real NULL for a missing id, so those rows were written by an *older* version of the function. The shipped database spans more than one generation of the code.

Dead weight in the schema and around it

The `filter` table is a complete model with a relationship, is created by `create_all`, and holds **0 rows** because no line of code ever touches it. Its job (remembering the day/week toggle) ended up in a plain Python attribute, `self.filter_mode`, which is never persisted.

`swap_id` and `remove_id` on `course` are the fossil of a design where a swap would *link* the new enrollment to the old rather than delete and re-insert. All NULL, verified. That design would not have had the bug in Section 3.5.

And `courses_database.py`, which the README calls “a seeding helper for the course catalog”, is nothing of the kind: it contains no course data and no seeding code. It is a second, incompatible `Student` model (its key is `id`, not `std_id`; its `dob` is a `String`, not a `Date`) that no module imports. Its `create_all` is commented out, which is the only reason it has never corrupted anything. Delete it, and fix that README line.

Passwords are plaintext, and importing anything writes to the database

`Student.password` is a plain `String` and login does `password == user_password`. The README already says so, in public, and lists `bcrypt` as the follow-up. That is the right way to carry it: do not defend it, own it.

Separately, `student_database.py` creates the engine, runs `create_all`, and opens a session *at import time*, with `echo=True` so every SQL statement the process ever emits is printed to the terminal. So importing any screen touches the database file and starts the spam. There is exactly one session for the whole process, shared by every screen, never scoped to a user action. That shared session is what makes the swap’s partial commit stick.

3 The enrollment rules

Four operations, each with its own rules, enforced at different layers and to very different standards.

3.1 The cart: choosing is not committing

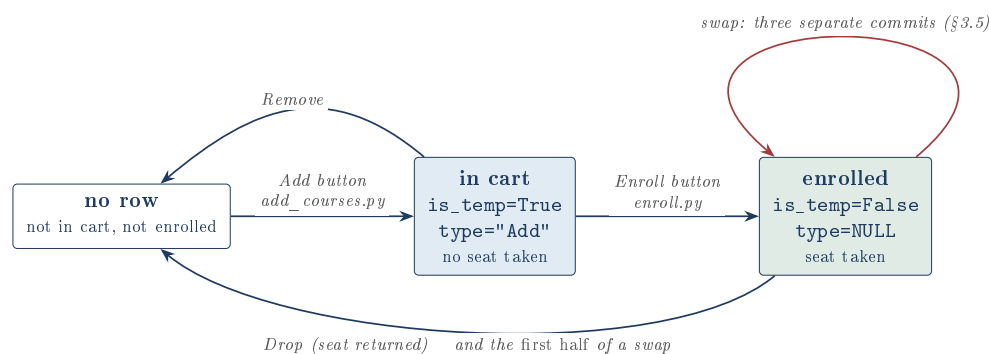


Figure 3: The two persisted states of an enrollment, and every transition between them. The red self-loop is the swap, and it is the one that is not safe.

The split is the right model: a cart can be abandoned, and it deliberately does not reserve a seat (the `capacity` call in `add_courses.py` is present but commented out). The cost is that a `course` row means two different things depending on a boolean, and every query has to remember to filter on it.

3.2 Rule 1: can this be added?

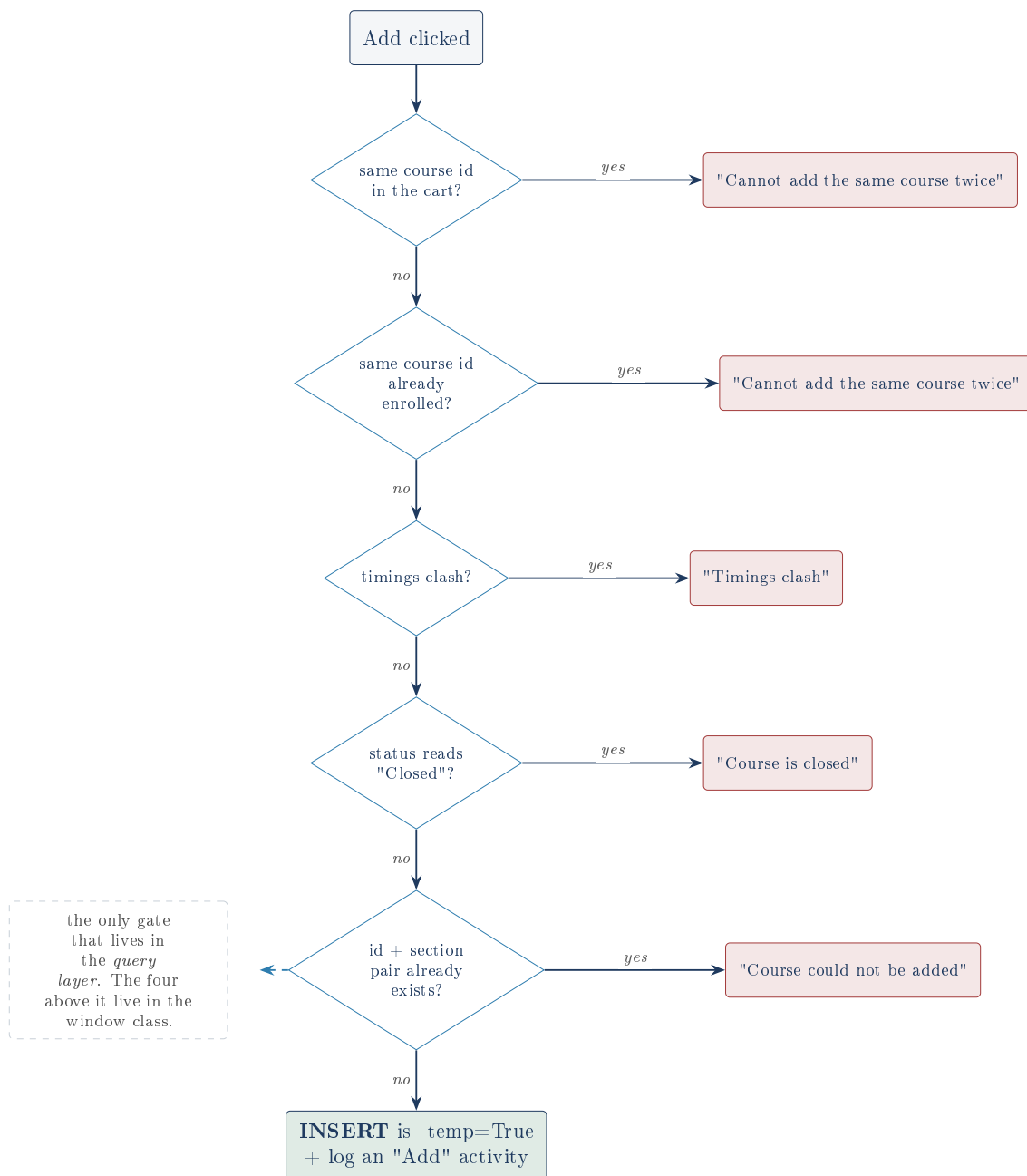


Figure 4: The add gate, in the order the code evaluates it.

The rules disagree with each other about what a duplicate is

The two top gates compare **course id only**, so 201/L1 in your cart blocks 201/L2. That is the intent: one section per course. But the bottom gate, `check_if_course_already_exist`, compares **id and section together**, and so would allow both. The database agrees with the bottom gate, because `course`'s primary key includes `section`.

So the rule “one section per course” is enforced in exactly one place, the weakest one: a window class. Nothing in the schema or the data layer encodes it, and anything inserting by another path, including `swap_courses`, bypasses it.

3.3 Rule 2: the clash detector

Times are reduced to seconds since midnight (`time_to_seconds`), and the `days` string is sliced into two-character codes (`get_days_list("MoWe")` gives `['Mo', 'We']`). Then, for every course the student already holds, if the two share a day code, three interval tests run. They are exhaustive: any overlap hits at least one.

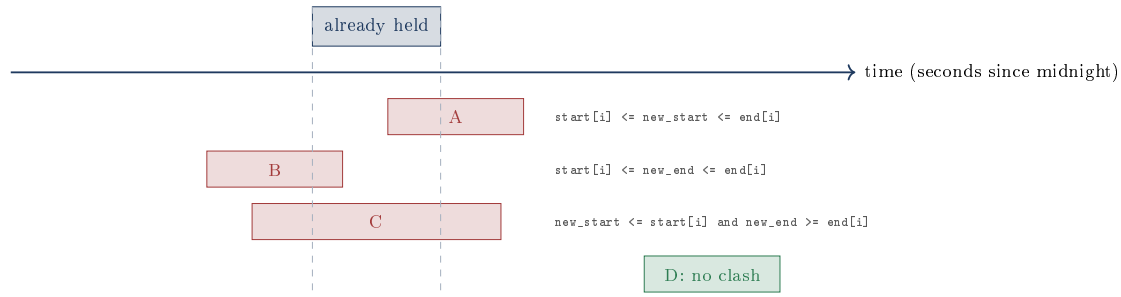


Figure 5: The three overlap cases tested against each already-held section, and only if the two share a day code.

Back-to-back classes are rejected

All three tests use `<=` on the boundary. A candidate starting at exactly 12:45, against a held course ending at exactly 12:45, satisfies test A and is refused as a clash. Look at the real catalogue: 201/L1 runs 11:30 to 12:45, and consecutive classes are the normal case in a university timetable. This rule forbids them. A strict `<` on the boundary comparisons is the fix. It is easy to trip over in a live demo.

The rule lives only in the UI, twice, and has already started to drift

`AllQueries.add_course` never consults the clash checker. The check exists as two near-identical 40-line copies, one in `add_courses.py` and one in `swap_courses.py`. A clash-free timetable is therefore a property the *windows* maintain, not something the data layer or the schema guarantees. And the two copies already call *different* query methods.

One of those methods is worth its own note:

```
get_days_timings_of_all_courses_by_id_temp_false
```

does **not** filter on `is_temp`, despite the name. Here the code is right and the name is wrong: when adding a third course you want it checked against your enrolled courses *and* the two already in your cart, or you could fill a cart with three mutually clashing courses and enroll in all of them. The fix is a rename, not a behaviour change. That is a better thing to say than “there’s a bug”.

3.4 Rule 3: capacity

current_capacity counts *empty* seats, not filled ones

The name works against you. Enrolling calls `update_current_capacity(..., "Subtract")`; dropping calls it with "Add". The number goes *down* as students join. It is remaining seats. The data agrees: untouched sections read 50 (equal to `max_capacity`), sections with three enrollments read 47. `status` is derived from it: false, meaning “Closed”, exactly when it hits zero, and the catalogue screens hide any closed section.

max_capacity is never read, and the shipped data is already over it

Verified by grep across every .py and .ui: the only occurrence of `max_capacity` in the entire project is the column declaration. The ceiling is not enforced; only the floor is.

This is not hypothetical. Two sections in the shipped `student.db`, 330/L3 and 310/L2, have `current_capacity = 51` against `max_capacity = 50`. Reproduced by running the real method against a copy:

```
1 201/L2 before:          max=50, current=50, status=True
2 after an ENROLL (Subtract): max=50, current=49, status=True
3 after two DROPS (Add, Add): max=50, current=51, status=True  <-- past the maximum
```

The fix belongs in the function that already has a “negative” branch: refuse to increment past `max_capacity`, and check *before* mutating rather than after. There is also no reservation and no locking, so two students could both take the last seat; for a single-user coursework demo that never happens, but it is the honest answer to “what breaks if two people use this at once”.

3.5 Rule 4: the swap, and the bug that matters most

The rules are simple: the two courses must be the same course id, and the sections must differ. So a swap moves you between sections of one course. The implementation is where it goes wrong.

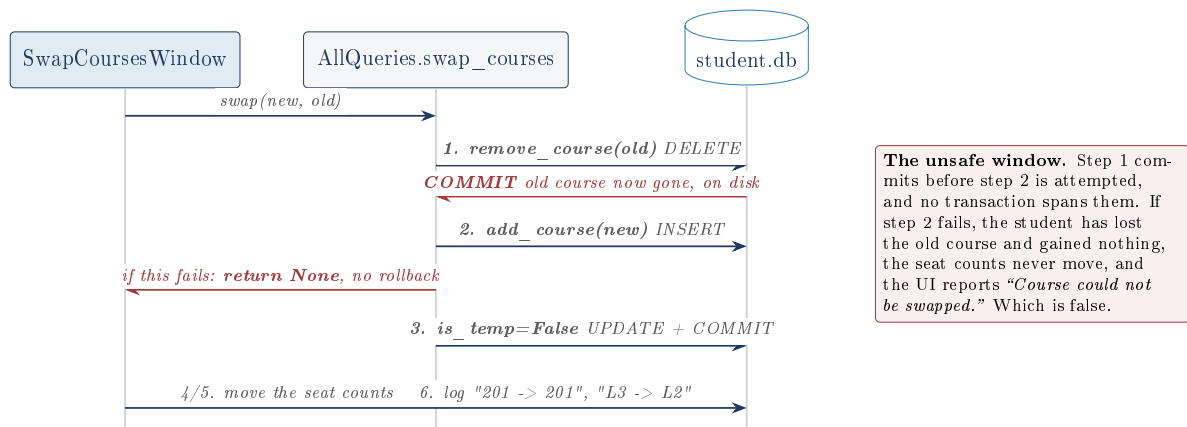


Figure 6: The swap, showing exactly where the atomicity is missing.

The swap is not atomic and destroys data. Verified by running it.

`remove_course` does its own unconditional `session.commit()`, outside the `try` that guards the add. Tested against a copy of the shipped database, asking for a swap onto a section that does not exist so step 2 must fail:

```
1 enrolled before: [(201,'L1'), (202,'S2'), (205,'L3'), (330,'L3'), (310,'L2')]
2 swap_courses(new=999/'ZZ' (nonexistent), old=201/'L1') -> returned: None
3 enrolled after:  [(202,'S2'), (205,'L3'), (330,'L3'), (310,'L2')]
4 LOST: [(201,'L1')]    GAINED: []
```

The enrollment was deleted and committed, the replacement never created, and the user was shown “Course could not be swapped.” The seat is not even returned, because the capacity steps sit below the early return.

The fix is small and worth being able to state instantly: wrap steps 1 to 3 in one transaction, do not commit inside `remove_course` when it is called from a swap, and `rollback` in the except. SQLAlchemy gives you this with `with session.begin():`.

The reason it was not done is visible in the code, and it is the real lesson: `remove_course`

and `add_course` were each written to be self-contained, committing operations for their own screens, and `swap_courses` reuses them as-is. **Composing two atomic operations does not give you an atomic operation.** That is the sentence to have ready, and it is the “A” in ACID, which is exactly what the course was about.

The dead `swap_id` / `remove_id` columns are what the original design was for: linking the new row to the old rather than delete-then-insert. That design would not have had this bug.

3.6 Rule 5: the drop, and one more curiosity

Dropping is unconditional: confirm, delete the row, return the seat, log it. There is no rule about whether you may.

An enroll-screen bug that cancels itself out

`enroll.py` tries to collect “only the ticked rows” with `if item.checkState():`. In PyQt5, `checkState()` returned an integer and `Unchecked` was 0, so this worked. PyQt6 replaced those constants with real Python enums, and **an enum member is truthy regardless of its value**. Verified by installing PyQt6 6.11 and asking: `bool(Qt.CheckState.Unchecked)` is `True`. The condition is true for every row.

It has no visible effect, for two compounding reasons. The line above it has already enrolled the entire cart in the database before the loop runs, and no code ever makes those items checkable, so no checkbox is ever shown. The intended feature does not exist in the UI, and the code that would have implemented it is broken in a way that happens to produce what the rest of the screen assumes: enroll everything.

Two wrongs make a right. It is a better story than a bug, because it traces a defect to the exact language semantics that caused it and then judges honestly that it does not matter today. What it does mean is that the dead code should go, because the day someone adds real checkboxes it will silently enroll the unticked ones.

4 The email flow, which is the best code here

Login is email plus a plaintext password compare, then a six-digit code in a second window. The interesting part is what happens when nobody has configured a mailbox.

Gate the network call, then pick a fallback that tells the truth

Anyone cloning this repository has no Gmail App Password, and a blind `smtplib` call would hang or throw. So both email paths check `ATTENDANCE_SENDER_EMAIL` and `ATTENDANCE_SENDER_PASSWORD` first, and each picks an honest fallback for its own context: `verification.py` prints the code to the terminal and says so, so login still completes; `attendance.py` returns `False` and the dialog reads “Attendance marked, but confirmation emails could not be sent” instead of claiming they went.

That second distinction is the whole point. The lazy version returns `True` and shows “emails sent!”. This code refuses to lie to the user about something it did not do. Two env vars drive both paths and `.env.example` documents them. This is better engineering than most coursework contains, and it is the reason the app runs for anyone who clones it.

But be precise about what the MFA is and is not

The code is generated with `random.randint` (not `secrets`), held as a string attribute on the window, and compared there. In a desktop app the user owns the process, so this is a faithful

demonstration of the MFA *flow*, not a security control. There is no expiry and no attempt limit. Say that rather than overselling it. Also, the env vars are named `ATTENDANCE_SENDER_*` and are used for login too, which is a naming debt a reviewer will ask about. And the Docker path can never send mail at all: verified by `grep`, neither the `Dockerfile` nor any run script forwards those two variables, and a container gets a fresh environment. It degrades gracefully, which vindicates the fallback design, but it means email is always off in a containerised demo. Adding `-e` flags to the `docker run` lines is a one-line fix per script.

5 The other screens, briefly

View schedule has two modes on one table. Day mode filters by `current_day[:2]`, taking the first two characters of `strftime("%A")` so “Monday” becomes “Mo”, matched against the same two-character day codes the clash detector uses. It depends on an English locale: on a translated system the day view silently shows nothing. Week mode hides the day column and lists everything. The Excel export builds a pandas DataFrame and, rather than clobbering an old file, probes with `read_excel` and bumps the name to `schedule_by_week (1).xlsx`. Nice thought, but it fully parses a spreadsheet to answer a question `os.path.exists` answers for free, and it only catches `FileNotFoundError`, so a file locked open in Excel is unhandled. Day-mode export is deliberately disabled because the hidden column misaligned the sheet: the right call, but since `filter_mode` starts at “Day”, **Export does nothing until you press Week first**, which looks like a broken button.

Attendance shows four hardcoded rows for four fixed dates in March 2025. There is no attendance table in the schema; verified, the database has exactly five tables and none of them is one. So marks never persist. Its docstring says it checks whether the status changed; it does not, and re-sends and re-logs all four rows on every press. The README already lists this under “What I’d do differently”, which is the right instinct.

Activity log reads every row for the student and renders it. No sorting, no pagination, and nothing ever prunes it; 116 rows today and growing, partly because the schedule window logs a “Filter by Day” activity in its *constructor*, before the user has done anything.

Two classes share one name

`enroll.py` defines `ViewScheduleWindow` (the checkout screen) and `view_schedule.py` defines `ViewScheduleWindow` (the schedule screen). Different files, different purposes, same name. Nothing breaks because no module imports both, but it is a trap for the first person who needs to. The history is visible: `enroll.ui` and `add_courses.ui` are both still named `view_schedule_window` internally, so three files carry the fingerprint of one original. Also orphaned: `add_remove_interaction.ui`, the largest layout in the project, loaded by nothing.

Running python queries.py wipes your data

Its `__main__` block calls `delete_all_data_activity_log()` and `delete_everything_in_course_table()`, which drop every row of activity and course for *every* student, with no confirmation. A development reset script that shipped inside the module.

6 Where the README and the code disagree

The README is unusually honest, and names most of the weaknesses above in public on a portfolio piece, which is the hardest thing on this list to do. Three claims do not survive contact with the code, and it is better to be the person who caught them.

Three corrections

1. “`courses_database.py` is a seeding helper for the course catalog.” It is not. It contains no course data and no seeding code: it is a dead, schema-incompatible second `Student` model that nothing imports.
2. “`AllQueries.glob_id` is set by whichever window ran last, so the query layer’s correctness depends on call order.” Read the code: `glob_id` is assigned in `__init__`, so it is an *instance* attribute. Every screen constructs its own `AllQueries` and sets its own id from its own email. Screens do not clobber each other, and the stated hazard does not exist as described. Two real hazards are nearby, though: the default `glob_id` of 20180 is Alice, a live student, so any `AllQueries()` built without a login silently acts as her (every screen also defaults its `email` to hers, which is how each module’s `__main__` block runs standalone with no password); and the *session* genuinely is a shared module global, which is what makes the swap’s partial commit durable.
3. **The described headless test harness is not in the repository.** The README details a specific methodology (`QT_QPA_PLATFORM=offscreen`, monkeypatched `QMessageBox` statics, login through every sub-screen) and says “this is how the project was last verified”. `git ls-files` shows no test file, no `tests/`, no pytest config, no CI. Most likely (**inferred**) it was a throwaway script never committed, which is a fine thing to have done, but the present tense reads as though it ships. Either commit the script or reword it. The README’s own closing bullet, “Add tests. Everything here was verified by clicking through the app”, sits oddly beside it, and the two should be reconciled.

How the facts here were established

Every file was read end to end. `student.db` was opened read-only and its schema, row counts and data inspected directly, so every number above came out of the file. `queries.py` was executed against a *copy* of that database to verify the capacity arithmetic and the swap; PyQt6 was installed in a throwaway environment to settle the `checkState` question. The GUI itself was never launched (PyQt6 is not installed on the documenting machine and the app needs a desktop), and the Docker image was never built (the daemon was not running), so claims in those two areas are labelled inferred where they appear. One such: the Dockerfile installs `libgl1-mesa-glx`, which was removed in Debian 12, the base that `python:3.10-slim` resolves to today, so the build probably fails now. Unverified. Check it before demoing; `libgl1` is the replacement.

7 The short version

The five things to know. The swap loses data and it is verified, not theoretical. The foreign keys are invalid and only work because SQLite never checks them. `max_capacity` is never read and the shipped data is already over it. Passwords are plaintext, and the README says so. The clash rule forbids back-to-back classes.

What is genuinely good, and worth leading with. The `AllQueries` choke point is a real architectural decision that paid off for a two-person team, and the code supports the story. The cart/enroll split is the right model. The env-gated email with a truthful fallback in both paths is better than most coursework, and it is why the app still runs for anyone who clones it. The Excel auto-increment shows someone thought about the second press of the button. Disabling the day export beat shipping a misaligned spreadsheet. And the README names most of its own flaws in public, which is the rarest thing here.

Glossary

Atomicity

An operation is atomic if it either fully happens or fully does not, with no observable in-between. Databases provide it with **transactions**: begin, make several changes, then **COMMIT** them as one indivisible act or **ROLLBACK** the lot. The ‘A’ in ACID, and the subject of Section 3.5.

Composite primary key

A primary key made of more than one column. Here, `course_list` is keyed by (`course_list_id`, `section`) and `course` by (`course_id`, `std_id`, `section`). Composite keys are the root of the foreign-key problem: only half of them was carried into the FK declarations.

Foreign key

A column pointing at another table’s primary key, expressing a relationship. A database can enforce it, refusing rows that point at nothing. SQLite does not, by default.

GUI / PyQt6 / Qt

A GUI is software you click on. Qt is a C++ toolkit that supplies the buttons, tables and windows; PyQt6 is the Python binding for Qt 6 that lets Python drive it.

Junction table

The third table needed to express a many-to-many relationship (many students per section, many sections per student). Here it is `course`.

MFA / SMTP / App Password

MFA proves identity with more than one kind of evidence: a password (something you know) plus access to a mailbox (something you have). SMTP is the mail-sending protocol, spoken by Python’s `smtplib`. Gmail refuses a normal password from a script, so you generate an App Password, a separate secret scoped to one application.

Normalisation

Organising a schema so each fact is stored once. This catalogue is *not* normalised: a course name repeats on every one of its section rows.

ORM / SQLAlchemy

An Object-Relational Mapper translates database rows into Python objects and back, so you write `session.query(Student).filter_by(...)` instead of SQL. SQLAlchemy is the standard Python one.

pandas / DataFrame / openpyxl

pandas is Python’s tabular-data library; its `DataFrame` is a table with named columns; `to_excel` writes one to a real `.xlsx`, delegating the format to `openpyxl` (which is why `openpyxl` is a dependency no line of code imports).

Qt Designer / .ui file

A visual editor for drawing windows, which saves them as XML `.ui` files. `uic.loadUi` reads one at runtime and builds the widgets, attaching each to the window under its Designer name. That is why `self.login_button` exists though no Python created it.

Signals and slots

Qt’s event mechanism. An object emits a signal (“I was closed”) without knowing who listens; connected functions (slots) are called. It is what decouples the child windows from the hub.

SQLite

A database that lives in one ordinary file with no server to install. Ideal for shipping a demo with its data already in it; not for many simultaneous users.

X11 forwarding

A container has no screen. On Linux the display is managed by a server called X11 listening on a socket; the run scripts mount that socket into the container and pass `DISPLAY`, so the containerised app draws on your desktop.